

Xây dựng trực tuyến cây hậu tố

Esko Ukkonen

Nguồn gốc: On-line construction of suffix trees

English Materials/Suffix Tree with Figs.pdf

Abstract

Bài viết trình bày một thuật toán trực tuyến để xây dựng cây hậu tố của một xâu cho trước trong thời gian tuyến tính theo độ dài xâu. Thuật toán mới có tính chất mong muốn là xử lý xâu từng ký hiệu một từ trái sang phải, và tại mọi thời điểm luôn có sẵn cây hậu tố của phần xâu đã quét. Phương pháp được phát triển như một phiên bản thời gian tuyến tính của một thuật toán rất đơn giản cho trie hậu tố, vốn có kích thước bậc hai. Dù thuật toán trie này có trường hợp xấu nhất bậc hai, nó vẫn có thể là một phương pháp thực tế tốt khi xâu không quá dài. Một biến thể khác của phương pháp cũng dẫn một cách tự nhiên tới các thuật toán đã biết để xây dựng ô-tô-mát hậu tố, hay DAWG.

Từ khóa: thuật toán thời gian tuyến tính, cây hậu tố, trie hậu tố, ô-tô-mát hậu tố, DAWG.

1 Giới thiệu

Cây hậu tố là một cấu trúc dữ liệu giống trie biểu diễn tất cả hậu tố của một xâu. Cây hậu tố đóng vai trò trung tâm trong nhiều thuật toán trên xâu, chẳng hạn [3, 7, 2]. Tuy nhiên, các thuật toán xây dựng cây hậu tố thời gian tuyến tính trong tài liệu thường bị xem là khá khó nắm bắt.

Mục đích chính của bài viết này là phát triển một cách xây dựng cây hậu tố dễ hiểu, dựa trên một ý tưởng tự nhiên giúp hoàn thiện bức tranh về cây hậu tố. Thuật toán mới có tính chất quan trọng là trực tuyến: nó xử lý xâu từng ký hiệu một từ trái sang phải, và luôn duy trì cây hậu tố của phần đã quét. Thuật toán dựa trên quan sát đơn giản rằng các hậu tố của xâu

$$T^i = t_1 \cdots t_i$$

có thể thu được từ các hậu tố của

$$T^{i-1} = t_1 \cdots t_{i-1}$$

bằng cách nối ký hiệu t_i vào cuối mỗi hậu tố của T^{i-1} , rồi thêm hậu tố rỗng. Các hậu tố của toàn bộ xâu $T = T^n = t_1 t_2 \cdots t_n$ vì vậy có thể thu được bằng cách lần lượt mở rộng hậu tố của T^0 thành hậu tố của T^1 , rồi tiếp tục cho tới khi hậu tố của T được tạo từ hậu tố của T^{n-1} .

Cách nhìn này khác với phương pháp của Weiner [13], vốn đi từ phải sang trái và thêm các hậu tố vào cây theo thứ tự tăng dần của độ dài, bắt đầu từ hậu tố ngắn nhất; nó cũng khác với phương pháp của McCreight [9], vốn thêm các hậu tố theo thứ tự giảm dần của độ dài. Tuy nhiên, dù trực giác ban đầu khác nhau rõ rệt, thuật toán ở đây và thuật toán của McCreight trong dạng cuối cùng lại khá gần nhau về mặt chức năng.

Thuật toán của chúng ta được hiểu tốt nhất như một phiên bản thời gian tuyến tính của một thuật toán khác trong [12] cho trie hậu tố có kích thước bậc hai. Thuật toán rất cơ bản đó, giống thuật toán cây vị trí

trong [8], được trình bày ở Mục 2. Nó không chạy trong thời gian tuyến tính: thời gian tỷ lệ với kích thước trie hậu tố, vốn có thể là bậc hai. Tuy nhiên, một sửa đổi khá trong suốt, được mô tả ở Mục 4, cho ta phương pháp trực tuyến thời gian tuyến tính cho cây hậu tố, đồng thời đem lại một góc nhìn tự nhiên giúp hiểu phép xây dựng này.

Mục 5 chỉ ra thêm rằng trie hậu tố được bổ sung các liên kết hậu tố cho ta một đặc trưng cơ bản của ô-tô-mát hậu tố, còn gọi là đồ thị từ phi chu trình có hướng, hay DAWG. Điều này lập tức dẫn tới một thuật toán xây dựng các ô-tô-mát đó. Phương pháp thu được về bản chất trùng với các thuật toán đã biết trong [4, 5, 6]. Một lần nữa, cách nhìn mới là tự nhiên và giúp hiểu rõ các phép xây dựng ô-tô-mát hậu tố.

2 Xây dựng trie hậu tố

Cho $T = t_1 t_2 \cdots t_n$ là một xâu trên bảng chữ cái Σ . Một xâu x được gọi là xâu con của T nếu $T = uxv$ với một số xâu, có thể rỗng, u và v . Một xâu

$$T_i = t_i \cdots t_n, \quad 1 \leq i \leq n + 1,$$

là một hậu tố của T ; đặc biệt, $T_{n+1} = \epsilon$ là hậu tố rỗng. Tập mọi hậu tố của T được ký hiệu là $\sigma(T)$. Trie hậu tố của T là trie biểu diễn $\sigma(T)$.

Chính xác hơn, ta ký hiệu trie hậu tố của T là

$$\text{STrie}(T) = (Q \cup \{\perp\}, \text{root}, F, g, f).$$

Đây là một ô-tô-mát hữu hạn đơn định được tăng cường: đồ thị chuyển trạng thái của nó có dạng cây và biểu diễn trie của $\sigma(T)$, còn phần tăng cường là hàm hậu tố f và trạng thái phụ $\perp$. Tập trạng thái Q của $\text{STrie}(T)$ có thể đặt tương ứng một-một với các xâu con của T . Trạng thái ứng với xâu con x được ký hiệu là $\bar{x}$.

Trạng thái đầu root ứng với xâu rỗng ϵ , và tập trạng thái kết thúc F ứng với $\sigma(T)$. Hàm chuyển g được định nghĩa bởi

$$g(\bar{x}, a) = \bar{y}$$

với mọi $\bar{x}, \bar{y} \in Q$ sao cho $y = xa$, trong đó $a \in \Sigma$.

Hàm hậu tố f được định nghĩa cho mỗi trạng thái $\bar{x} \in Q$ như sau. Nếu $\bar{x} \neq \text{root}$ thì $x = ay$ với một ký hiệu $a \in \Sigma$, và ta đặt

$$f(\bar{x}) = \bar{y}.$$

Ngoài ra, $f(\text{root}) = \perp$.

Trạng thái phụ $\perp$ cho phép ta viết các thuật toán phía sau mà không cần tách riêng hậu tố rỗng và hậu tố không rỗng, hay tách riêng root và các trạng thái khác. Trạng thái $\perp$ được nối với trie bởi

$$g(\perp, a) = \text{root}$$

với mọi $a \in \Sigma$. Ta không định nghĩa $f(\perp)$. Có thể xem các chuyển từ $\perp$ tới root là nhất quán với các chuyển còn lại: trạng thái $\perp$ ứng với nghịch đảo a^{-1} của mọi ký hiệu $a \in \Sigma$; vì $a^{-1}a = \epsilon$, ta đặt $g(\perp, a) = \text{root}$ do root ứng với ϵ .

Theo thuật ngữ của McCreight [9], $f(r)$ được gọi là **liên kết hậu tố** của trạng thái r . Các liên kết hậu tố sẽ được dùng trong quá trình xây dựng cây hậu tố; chúng cũng có nhiều ứng dụng khác [11, 12].

Ô-tô-mát $\text{STrie}(T)$ trùng với ô-tô-mát khớp xâu Aho–Corasick [1] cho tập từ khóa $\{T_i \mid 1 \leq i \leq n + 1\}$; trong [1], liên kết hậu tố được gọi là chuyển thất bại.

Hình 1 của bài gốc minh họa quá trình xây dựng STrie(cacao): các cung chuyển trạng thái được vẽ đậm, còn các chuyển thất bại, tức liên kết hậu tố, được vẽ mảnh. Để hình dễ đọc, chỉ hai lớp cuối của các liên kết hậu tố được vẽ tường minh.

Dễ xây dựng STrie(T) trực tuyến bằng một lượt quét trái sang phải trên T . Ký hiệu $T^i = t_1 \cdots t_i$ là tiền tố của T , với $0 \leq i \leq n$. Các kết quả trung gian của phép xây dựng là STrie(T^i) với $i = 0, 1, \dots, n$.

Quan sát then chốt để nhận STrie(T^i) từ STrie(T^{i-1}) là các hậu tố của T^i có thể thu được bằng cách nối t_i vào cuối mỗi hậu tố của T^{i-1} , rồi thêm một hậu tố rỗng:

$$\sigma(T^i) = \sigma(T^{i-1})t_i \cup \{\epsilon\}.$$

Theo định nghĩa, STrie(T^{i-1}) chấp nhận $\sigma(T^{i-1})$. Để nó chấp nhận $\sigma(T^i)$, ta xét tập trạng thái kết thúc F_{i-1} của STrie(T^{i-1}). Nếu $r \in F_{i-1}$ chưa có chuyển theo t_i , ta thêm một chuyển như vậy từ r tới một trạng thái mới, trạng thái này trở thành lá mới của trie. Các trạng thái mà một chuyển cũ hoặc mới theo t_i từ một trạng thái trong F_{i-1} đi tới, cùng với *root*, tạo thành tập trạng thái kết thúc F_i của STrie(T^i).

Các trạng thái $r \in F_{i-1}$ cần nhận chuyển mới có thể được tìm bằng liên kết hậu tố. Định nghĩa của hàm hậu tố cho thấy $r \in F_{i-1}$ khi và chỉ khi

$$r = f^j(t_1 \cdots t_{i-1})$$

với một $0 \leq j \leq i-1$. Vì vậy mọi trạng thái trong F_{i-1} nằm trên đường đi liên kết hậu tố bắt đầu từ trạng thái sâu nhất $\overline{t_1 \cdots t_{i-1}}$ của STrie(T^{i-1}) và kết thúc ở $\perp$. Ta gọi đường đi quan trọng này là **đường biên** của STrie(T^{i-1}).

Ta duyệt đường biên. Nếu một trạng thái $\bar{z}$ trên đường biên chưa có chuyển theo t_i , ta thêm trạng thái mới $\overline{zt_i}$ và chuyển mới

$$g(\bar{z}, t_i) = \overline{zt_i}.$$

Như vậy g được cập nhật. Để cập nhật f , các trạng thái mới $\overline{zt_i}$ được nối với nhau bằng các liên kết hậu tố mới, tạo thành một đường đi bắt đầu từ trạng thái $\overline{t_1 \cdots t_i}$. Rõ ràng đây chính là đường biên của STrie(T^i).

Việc duyệt F_{i-1} dọc theo đường biên có thể dừng ngay khi gặp trạng thái đầu tiên $\bar{z}$ sao cho trạng thái $\overline{zt_i}$, và do đó chuyển $g(\bar{z}, t_i) = \overline{zt_i}$, đã tồn tại. Thật vậy, nếu $\overline{zt_i}$ đã là một trạng thái thì STrie(T^{i-1}) phải chứa trạng thái $\overline{z't_i}$ và chuyển $g(\bar{z}', t_i) = \overline{z't_i}$ với mọi $z' = f^j(\bar{z})$, $j \geq 1$. Nói cách khác, nếu zt_i là xâu con của T^{i-1} thì mọi hậu tố của zt_i cũng là xâu con của T^{i-1} . Chú ý rằng trạng thái $\bar{z}$ như vậy luôn tồn tại, vì $\perp$ là trạng thái cuối của đường biên và $\perp$ có chuyển cho mọi ký hiệu t_i .

Khi phép duyệt dừng như vậy, thủ tục tạo một trạng thái mới cho mỗi liên kết hậu tố đã xét trong lúc duyệt. Do đó toàn bộ thủ tục tốn thời gian tỷ lệ với kích thước ô-tô-mát thu được.

Thuật toán 1. Xây dựng STrie(T^i) từ STrie(T^{i-1}). Ở đây top là trạng thái $\overline{t_1 \cdots t_{i-1}}$.

```

r <- top;
while g(r, ti) is undefined do
  create new state r' and new transition g(r, ti) = r';
  if r != top then create new suffix link f(olldr') = r';
  oldr' <- r';
  r <- f(r);
create new suffix link f(olldr') = g(r, ti);
top <- g(top, ti).
```

Bắt đầu từ STrie(ϵ), gồm chỉ *root*, $\perp$ và các liên kết giữa chúng, rồi lặp Thuật toán 1 cho $t_i = t_1, t_2, \dots, t_n$, ta thu được STrie(T). Thuật toán là tối ưu theo nghĩa thời gian chạy tỷ lệ với kích thước kết quả cuối cùng STrie(T). Kích thước này lại tỷ lệ với $|Q|$, tức số xâu con khác nhau của T . Đáng tiếc, số đó có thể là bậc hai theo $|T|$, chẳng hạn khi $T = a^n b^n$.

Định lý 1 Trie hậu tố $\text{STrie}(T)$ có thể được xây dựng trong thời gian tỷ lệ với kích thước của $\text{STrie}(T)$; trong trường hợp xấu nhất, kích thước này là $O(|T|^2)$.

3 Cây hậu tố

Cây hậu tố $\text{STree}(T)$ của T là cấu trúc dữ liệu biểu diễn $\text{STrie}(T)$ trong không gian tuyến tính theo độ dài $|T|$. Điều này đạt được bằng cách chỉ biểu diễn một tập con $Q' \cup \{\perp\}$ của các trạng thái trong $\text{STrie}(T)$. Ta gọi các trạng thái trong $Q' \cup \{\perp\}$ là **trạng thái tường minh**. Tập Q' gồm mọi trạng thái rẽ nhánh, tức các trạng thái có ít nhất hai chuyển đi ra, và mọi lá, tức các trạng thái không có chuyển đi ra, của $\text{STrie}(T)$. Theo định nghĩa, *root* được xem là trạng thái rẽ nhánh. Các trạng thái còn lại của $\text{STrie}(T)$, tức các trạng thái khác *root* và $\perp$ có đúng một chuyển đi ra, được gọi là **trạng thái ngầm** khi xét như trạng thái của $\text{STree}(T)$; chúng không xuất hiện tường minh trong $\text{STree}(T)$.

Giả sử xâu w được đọc trên đường chuyển trong $\text{STrie}(T)$ giữa hai trạng thái tường minh s và r . Trong $\text{STree}(T)$, đường đó được biểu diễn bởi một chuyển tổng quát

$$g'(s, w) = r.$$

Để tiết kiệm không gian, xâu w thật ra được biểu diễn bởi một cặp con trỏ (k, p) vào T , gọi là con trỏ trái và con trỏ phải, sao cho $t_k \cdots t_p = w$. Vì vậy chuyển tổng quát có dạng

$$g'(s, (k, p)) = r.$$

Các con trỏ như vậy tồn tại vì phải có một hậu tố T_i sao cho đường chuyển của T_i trong $\text{STrie}(T)$ đi qua s và r . Ta có thể chọn i nhỏ nhất như vậy, rồi cho k và p trỏ tới đoạn con của T_i được đọc trên đường từ s tới r . Một chuyển $g'(s, (k, p)) = r$ được gọi là chuyển theo a nếu $t_k = a$. Mỗi s có nhiều nhất một chuyển theo a với mỗi $a \in \Sigma$.

Các chuyển $g(\perp, a) = \text{root}$ được biểu diễn tương tự. Nếu $\Sigma = \{a_1, a_2, \dots, a_m\}$ thì $g(\perp, a_j) = \text{root}$ được biểu diễn bởi

$$g'(\perp, (-j, -j)) = \text{root}, \quad j = 1, \dots, m.$$

Do đó cây hậu tố $\text{STree}(T)$ có hai thành phần: bản thân cây và xâu T . Cấu trúc này có kích thước tuyến tính theo $|T|$, vì Q' có nhiều nhất $|T|$ lá, mỗi hậu tố không rỗng cho nhiều nhất một lá, và do đó có nhiều nhất $|T| - 1$ trạng thái rẽ nhánh khi $|T| > 1$. Số chuyển giữa các trạng thái trong Q' vì thế nhiều nhất là $2|T| - 2$, và mỗi chuyển chiếm không gian hằng số nhờ dùng con trỏ thay vì lưu xâu tường minh. Ở đây ta giả sử mô hình RAM chuẩn, trong đó một con trỏ chiếm không gian hằng số.

Ta lại tăng cường cấu trúc bằng hàm hậu tố f' , lần này chỉ định nghĩa cho các trạng thái rẽ nhánh $\bar{x} \neq \text{root}$ bởi

$$f'(\bar{x}) = \bar{y},$$

trong đó y là một trạng thái rẽ nhánh và $x = ay$ với một $a \in \Sigma$; ngoài ra $f'(\text{root}) = \perp$. Hàm f' là xác định tốt: nếu $\bar{x}$ là trạng thái rẽ nhánh thì $f'(\bar{x})$ cũng là trạng thái rẽ nhánh. Các liên kết hậu tố này được biểu diễn tường minh. Đôi khi ta cũng sẽ nói tới liên kết hậu tố ngầm, tức các liên kết tường tượng giữa các trạng thái ngầm.

Ta ký hiệu cây hậu tố của T là

$$\text{STree}(T) = (Q' \cup \{\perp\}, \text{root}, g', f').$$

Một trạng thái tường minh hoặc ngầm r của cây hậu tố được tham chiếu bằng một **cặp tham chiếu** (s, w) , trong đó s là một trạng thái tường minh tổ tiên của r , còn w là xâu đọc được trên các chuyển từ s tới r

trong trie hậu tố tương ứng. Cặp tham chiếu là **chuẩn tắc** nếu s là tổ tiên tường minh gần nhất của r , và vì vậy w ngắn nhất có thể. Với một trạng thái tường minh r , cặp tham chiếu chuẩn tắc hiển nhiên là (r, ϵ) . Như trên, xâu w được biểu diễn bởi cặp con trỏ (k, p) sao cho $t_k \cdots t_p = w$. Nhờ vậy cặp tham chiếu (s, w) có dạng $(s, (k, p))$. Cặp (s, ϵ) được biểu diễn là $(s, (p + 1, p))$.

Về mặt kỹ thuật, sẽ tiện hơn nếu bỏ các trạng thái kết thúc khởi định nghĩa cây hậu tố. Khi một ứng dụng cần trạng thái kết thúc tường minh, ta nhận chúng miễn phí bằng cách thêm vào T một ký hiệu kết thúc không xuất hiện ở nơi nào khác trong T . Các lá của cây hậu tố cho xâu như vậy tương ứng một-một với các hậu tố của T và tạo thành tập trạng thái kết thúc. Một cách khác là duyệt đường liên kết hậu tố từ lá $\tilde{T}$ tới $root$ và làm mọi trạng thái trên đường đó trở thành tường minh; đó chính là các trạng thái kết thúc của $STree(T)$. Trong nhiều ứng dụng của $STree(T)$, vị trí bắt đầu của mỗi hậu tố được lưu cùng trạng thái tương ứng. Cây tăng cường như vậy có thể dùng làm chỉ mục để tìm bất kỳ xâu con nào của T .

4 Xây dựng trực tuyến cây hậu tố

Thuật toán xây dựng $STree(T)$ sẽ mô phỏng Thuật toán 1. Phần lớn việc cần làm là rõ ràng. Hình 2 của bài gốc minh họa các pha xây dựng $STree(cacao)$; để đơn giản, hình vẽ ghi tường minh xâu gắn với mỗi chuyển. Tuy nhiên, để nhận được thuật toán thời gian tuyến tính, một số chi tiết cần được xem xét cẩn thận hơn.

Trước hết ta mô tả chính xác hơn Thuật toán 1 làm gì. Gọi

$$s_1 = \overline{t_1 \cdots t_{i-1}}, \quad s_2, s_3, \dots, s_i = root, \quad s_{i+1} = \perp$$

là các trạng thái của $STrie(T^{i-1})$ trên đường biên. Gọi j là chỉ số nhỏ nhất sao cho s_j không phải lá, và gọi j' là chỉ số nhỏ nhất sao cho $s_{j'}$ có chuyển theo t_i . Vì s_1 là lá còn $\perp$ là trạng thái không phải lá có chuyển theo t_i , cả j và j' đều xác định tốt và $j \leq j'$.

Bổ đề 1 *Thuật toán 1 thêm vào $STrie(T^{i-1})$ một chuyển theo t_i cho mỗi trạng thái s_h , $1 \leq h < j'$, sao cho với $1 \leq h < j$, chuyển mới mở rộng một nhánh cũ của trie đang kết thúc ở lá s_h , còn với $j \leq h < j'$, chuyển mới khởi đầu một nhánh mới từ s_h . Thuật toán 1 không tạo chuyển nào khác.*

Ta gọi trạng thái s_j là **điểm hoạt động** và $s_{j'}$ là **điểm kết thúc** của $STrie(T^{i-1})$. Các trạng thái này cũng hiện diện, tường minh hoặc ngầm, trong $STree(T^{i-1})$. Ví dụ, trong ba cây cuối của Hình 2 bài gốc cho xâu *cacao*, các điểm hoạt động lần lượt là $(root, c)$, $(root, ca)$ và $(root, \epsilon)$.

Bổ đề 1 nói rằng Thuật toán 1 chèn hai nhóm chuyển theo t_i vào $STrie(T^{i-1})$:

1. Các trạng thái trên đường biên đứng trước điểm hoạt động s_j nhận một chuyển. Chúng là lá, nên mỗi chuyển như vậy chỉ mở rộng một nhánh đã có của trie.
2. Các trạng thái từ điểm hoạt động s_j tới điểm kết thúc $s_{j'}$, không kể điểm kết thúc, nhận một chuyển mới. Chúng không phải lá, nên mỗi chuyển mới khởi đầu một nhánh mới.

Ta diễn giải điều này trên cây hậu tố $STree(T^{i-1})$. Nhóm chuyển thứ nhất, những chuyển mở rộng nhánh đã có, có thể được cài đặt bằng cách cập nhật con trỏ phải của mỗi chuyển biểu diễn nhánh đó. Giả sử

$$g'(s, (k, i - 1)) = r$$

là một chuyển như vậy. Con trỏ phải phải trở tới vị trí cuối $i - 1$ của T^{i-1} , vì r là lá và do đó đường đi tới r phải đọc ra một hậu tố của T^{i-1} không xuất hiện ở nơi khác trong T^{i-1} . Chuyển sau cập nhật là $g'(s, (k, i)) = r$. Việc này chỉ làm xâu trên chuyển dài hơn, không thay đổi các trạng thái s và r . Nếu cập nhật tường minh mọi chuyển như vậy thì tốn quá nhiều thời gian, nên ta dùng mẹo sau.

Mọi chuyển của $\text{STree}(T^{i-1})$ đi tới một lá được gọi là **chuyển mở**. Chuyển như vậy có dạng $g'(s, (k, i-1)) = r$, trong đó con trở phải buộc phải trở tới vị trí cuối $i-1$ của T^{i-1} . Vì vậy không cần biểu diễn giá trị thật của con trở phải. Thay vào đó, chuyển mở được biểu diễn là

$$g'(s, (k, \infty)) = r,$$

trong đó ∞ cho biết chuyển này “mở để lớn lên”. Thực chất, $g'(s, (k, \infty)) = r$ biểu diễn một nhánh có độ dài bất kỳ từ trạng thái s tới trạng thái tường tượng r ở vô cực. Khi chèn t_i vào nhánh này, ta không cần cập nhật tường minh con trở phải. Sau khi hoàn tất $\text{STree}(T)$, các ký hiệu ∞ có thể được thay bằng $n = |T|$. Như vậy nhóm chuyển thứ nhất được cài đặt mà không gây thay đổi tường minh nào trên $\text{STree}(T^{i-1})$.

Còn lại là cách thêm nhóm chuyển thứ hai vào $\text{STree}(T^{i-1})$. Chúng tạo các nhánh hoàn toàn mới bắt đầu từ các trạng thái $s_h, j \leq h < j'$. Việc tìm các trạng thái s_h cần cẩn thận, vì ở thời điểm đó chúng không nhất thiết là trạng thái tường minh. Chúng sẽ được tìm dọc đường biên của $\text{STree}(T^{i-1})$ bằng cặp tham chiếu và liên kết hậu tố.

Đặt $h = j$ và giả sử (s, w) là cặp tham chiếu chuẩn tắc cho s_h , tức điểm hoạt động. Vì s_h nằm trên đường biên của $\text{STree}(T^{i-1})$, w phải là một hậu tố của T^{i-1} . Do đó

$$(s, w) = (s, (k, i-1))$$

với một $k \leq i$.

Ta muốn tạo một nhánh mới bắt đầu từ trạng thái được biểu diễn bởi $(s, (k, i-1))$. Tuy nhiên, trước hết phải kiểm tra liệu $(s, (k, i-1))$ đã tham chiếu tới điểm kết thúc $s_{j'}$ hay chưa. Nếu có thì đã xong. Nếu chưa, cần tạo một nhánh mới. Khi đó trạng thái s_h được tham chiếu bởi $(s, (k, i-1))$ phải trở thành tường minh. Nếu nó chưa tường minh, ta tạo một trạng thái tường minh, vẫn ký hiệu là s_h , bằng cách tách chuyển chứa trạng thái ngầm tương ứng. Sau đó tạo một chuyển theo t_i từ s_h . Chuyển này phải là chuyển mở

$$g'(s_h, (i, \infty)) = s'_{j'}$$

trong đó $s'_{j'}$ là một lá mới. Ngoài ra, liên kết hậu tố $f'(s_h)$ được thêm nếu s_h vừa được tạo bằng cách tách chuyển.

Tiếp theo phép xây dựng chuyển sang s_{h+1} . Nếu cặp tham chiếu cho s_h là $(s, (k, i-1))$, thì cặp tham chiếu chuẩn tắc cho s_{h+1} là

$$\text{canonize}(f'(s), (k, i-1)).$$

Ở đây **CANONIZE** làm cặp tham chiếu trở thành chuẩn tắc bằng cách cập nhật trạng thái và con trở trái; con trở phải $i-1$ không đổi trong quá trình chuẩn tắc hóa. Các thao tác trên được lặp lại cho s_{h+1} , rồi tiếp tục cho tới khi tìm thấy điểm kết thúc $s_{j'}$.

Ta thu được thủ tục **UPDATE** dưới đây, biến $\text{STree}(T^{i-1})$ thành $\text{STree}(T^i)$ bằng cách chèn các chuyển theo t_i thuộc nhóm thứ hai. Thủ tục dùng **CANONIZE** vừa nêu, và thủ tục **TEST-AND-SPLIT** để kiểm tra một cặp tham chiếu có chỉ tới điểm kết thúc hay không. Nếu không, thủ tục tạo và trả về một trạng thái tường minh cho cặp tham chiếu đó, trừ khi cặp đã biểu diễn một trạng thái tường minh. Thủ tục **UPDATE** trả về một cặp tham chiếu cho điểm kết thúc $s_{j'}$; trên thực tế chỉ trả về trạng thái và con trở trái, vì con trở phải vẫn là $i-1$ cho mọi trạng thái trên đường biên.

Thủ tục UPDATE($s, (k, i)$). Giả sử $(s, (k, i-1))$ là cặp tham chiếu chuẩn tắc cho điểm hoạt động.

```

oldr <- root;
(end-point, r) <- test-and-split(s, (k, i - 1), ti);
while not end-point do
    create new transition g'(r, (i, infinity)) = r',

```

```

    where  $r'$  is a new state;
    if  $\text{oldr} \neq \text{root}$  then create new suffix link  $f'(\text{oldr}) = r$ ;
     $\text{oldr} \leftarrow r$ ;
     $(s, k) \leftarrow \text{canonize}(f'(s), (k, i - 1))$ ;
     $(\text{end-point}, r) \leftarrow \text{test-and-split}(s, (k, i - 1), t_i)$ ;
    if  $\text{oldr} \neq \text{root}$  then create new suffix link  $f'(\text{oldr}) = s$ ;
    return  $(s, k)$ .

```

Thủ tục TEST-AND-SPLIT kiểm tra trạng thái có cặp tham chiếu chuẩn tắc $(s, (k, p))$ có phải điểm kết thúc hay không, tức trong $\text{STrie}(T^{i-1})$ nó đã có chuyển theo t_i hay chưa. Ký hiệu t_i được truyền vào dưới tên t . Kết quả kiểm tra được trả về ở tham số thứ nhất. Nếu $(s, (k, p))$ không phải điểm kết thúc, trạng thái đó được làm tường minh, nếu cần bằng cách tách một chuyển, và trạng thái tường minh được trả về ở tham số thứ hai.

Thủ tục TEST-AND-SPLIT($s, (k, p), t$).

```

if  $k \leq p$  then
    let  $g'(s, (k', p')) = s'$  be the tk-transition from  $s$ ;
    if  $t = t_{\{k' + p - k + 1\}}$  then return (true,  $s$ )
    else
        replace the tk-transition above by transitions
             $g'(s, (k', k' + p - k)) = r$ 
             $g'(r, (k' + p - k + 1, p')) = s'$ ,
        where  $r$  is a new state;
        return (false,  $r$ )
else
    if there is no t-transition from  $s$  then return (false,  $s$ )
    else return (true,  $s$ ).

```

Thủ tục này tận dụng việc $(s, (k, p))$ là chuẩn tắc: câu trả lời cho phép kiểm tra điểm kết thúc có thể tìm trong thời gian hằng số bằng cách chỉ xét một chuyển từ s .

Thủ tục CANONIZE như sau. Với một cặp tham chiếu $(s, (k, p))$ cho một trạng thái r , thủ tục tìm và trả về trạng thái s' cùng con trở trái k' sao cho $(s', (k', p))$ là cặp tham chiếu chuẩn tắc cho r . Trạng thái s' là tổ tiên tường minh gần nhất của r , hoặc chính r nếu r tường minh. Vì vậy xâu đi từ s' tới r phải là một hậu tố của xâu $t_k \cdots t_p$ đi từ s tới r . Do đó con trở phải p không đổi, còn con trở trái k có thể trở thành $k' \geq k$.

Thủ tục CANONIZE($s, (k, p)$).

```

if  $p < k$  then return  $(s, k)$ 
else
    find the tk-transition  $g'(s, (k', p')) = s'$  from  $s$ ;
    while  $p' - k' \leq p - k$  do
         $k \leftarrow k + p' - k' + 1$ ;
         $s \leftarrow s'$ ;
        if  $k \leq p$  then
            find the tk-transition  $g'(s, (k', p')) = s'$  from  $s$ ;
    return  $(s, k)$ .

```

Để tiếp tục xây dựng cho ký hiệu văn bản kế tiếp t_{i+1} , cần tìm điểm hoạt động của $\text{STree}(T^i)$. Trước hết, s_j là điểm hoạt động của $\text{STree}(T^{i-1})$ khi và chỉ khi $s_j = \overline{t_j \cdots t_{i-1}}$, trong đó $t_j \cdots t_{i-1}$ là hậu tố dài

nhất của T^{i-1} xuất hiện ít nhất hai lần trong T^{i-1} . Tiếp theo, $s_{j'}$ là điểm kết thúc của $\text{STree}(T^{i-1})$ khi và chỉ khi $s_{j'} = \overline{t_{j'} \cdots t_{i-1}}$, trong đó $t_{j'} \cdots t_{i-1}$ là hậu tố dài nhất của T^{i-1} sao cho $t_{j'} \cdots t_{i-1} t_i$ là xâu con của T^{i-1} . Nhưng điều này nghĩa là nếu $s_{j'}$ là điểm kết thúc của $\text{STree}(T^{i-1})$ thì $t_{j'} \cdots t_{i-1} t_i$ là hậu tố dài nhất của T^i xuất hiện ít nhất hai lần trong T^i , tức trạng thái $g(s_{j'}, t_i)$ là điểm hoạt động của $\text{STree}(T^i)$.

Bổ đề 2 *Giả sử $(s, (k, i - 1))$ là một cặp tham chiếu của điểm kết thúc $s_{j'}$ của $\text{STree}(T^{i-1})$. Khi đó $(s, (k, i))$ là một cặp tham chiếu của điểm hoạt động của $\text{STree}(T^i)$.*

Thuật toán tổng thể để xây dựng $\text{STree}(T)$ như sau. Xâu T được xử lý từng ký hiệu một trong một lượt quét trái sang phải. Viết $\Sigma = \{t_{-1}, \dots, t_{-m}\}$ giúp trình bày các chuyển từ $\perp$ giống các chuyển khác.

Thuật toán 2. Xây dựng $\text{STree}(T)$ cho xâu $T = t_1 t_2 \cdots t_n$ trên bảng chữ cái $\Sigma = \{t_{-1}, \dots, t_{-m}\}$, trong đó $\perp$ là ký hiệu kết thúc không xuất hiện ở nơi khác trong T .

```
create states root and bottom;
for j <- 1, ..., m do
    create transition g' (bottom, (-j, -j)) = root;
create suffix link f' (root) = bottom;
s <- root; k <- 1; i <- 0;
while t_{i+1} !=  $\perp$  do
    i <- i + 1;
    (s, k) <- update(s, (k, i));
    (s, k) <- canonize(s, (k, i)).
```

Bước cuối trong mỗi vòng dựa trên Bổ đề 2: sau lời gọi UPDATE, cặp $(s, (k, i - 1))$ tham chiếu tới điểm kết thúc của $\text{STree}(T^{i-1})$, và do đó $(s, (k, i))$ tham chiếu tới điểm hoạt động của $\text{STree}(T^i)$.

Định lý 2 *Thuật toán 2 xây dựng trực tuyến cây hậu tố $\text{STree}(T)$ cho xâu $T = t_1 \cdots t_n$ trong thời gian $O(n)$.*

Chứng minh. Thuật toán xây dựng $\text{STree}(T)$ qua các cây trung gian

$$\text{STree}(T^0), \text{STree}(T^1), \dots, \text{STree}(T^n) = \text{STree}(T).$$

Nó là trực tuyến vì để xây dựng $\text{STree}(T^i)$, thuật toán chỉ cần truy cập i ký hiệu đầu tiên của T .

Để phân tích thời gian chạy, ta chia chi phí thành hai phần, và cả hai đều là $O(n)$. Phần thứ nhất là tổng thời gian của thủ tục CANONIZE. Phần thứ hai gồm phần còn lại: thời gian lặp lại việc đi theo đường liên kết hậu tố từ điểm hoạt động hiện tại tới điểm kết thúc, tạo các nhánh mới bằng UPDATE, rồi tìm điểm hoạt động kế tiếp bằng cách đi một chuyển từ điểm kết thúc, tức bước cuối của Thuật toán 2. Ta gọi các trạng thái, hay các cặp tham chiếu, trên các đường này là **trạng thái được thăm**.

Phần thứ hai tốn thời gian tỷ lệ với tổng số trạng thái được thăm, vì các thao tác tại mỗi trạng thái như tạo một trạng thái tường minh và nhánh mới, đi theo một liên kết hậu tố tường minh hoặc ngầm, và kiểm tra điểm kết thúc đều có thể cài đặt trong thời gian hằng số khi loại trừ CANONIZE. Nói chính xác hơn, điều này cũng cần giả thiết $|\Sigma|$ bị chặn độc lập với n . Gọi r_i là điểm hoạt động của $\text{STree}(T^i)$ với $0 \leq i \leq n$. Các trạng thái được thăm giữa r_{i-1} và r_i nằm trên một đường gồm một số liên kết hậu tố và một chuyển theo t_i . Đi theo một liên kết hậu tố làm giảm độ sâu, tức độ dài xâu đọc được trên đường từ $root$, đi một đơn vị; còn đi theo chuyển t_i làm tăng nó một đơn vị. Do đó số trạng thái được thăm trên đường, tính cả r_{i-1} nhưng không tính r_i , là

$$\text{depth}(r_{i-1}) - \text{depth}(r_i) + 2.$$

Tổng số trạng thái được thăm là

$$\sum_{i=1}^n (\text{depth}(r_{i-1}) - \text{depth}(r_i) + 2) = \text{depth}(r_0) - \text{depth}(r_n) + 2n \leq 2n.$$

Vì vậy phần thời gian thứ hai là $O(n)$.

Thời gian của mỗi lần gọi `CANONIZE` bị chặn bởi $a + bq$, với a, b là hằng số và q là số lần thực hiện thân vòng lặp trong `CANONIZE`. Tổng thời gian của `CANONIZE` vì vậy bị chặn tỷ lệ với tổng số lời gọi `CANONIZE` cộng với tổng số lần thực hiện thân vòng lặp trong mọi lời gọi. Có $O(n)$ lời gọi, vì mỗi trạng thái được thăm gây ra một lời gọi, hoặc ở trong `UPDATE`, hoặc trực tiếp ở bước cuối của Thuật toán 2. Mỗi lần thực hiện thân vòng lặp xóa một xâu không rỗng khỏi đầu trái của xâu $w = t_k \cdots t_p$ được biểu diễn bởi các con trỏ trong cặp tham chiếu $(s, (k, p))$. Trong toàn bộ quá trình, w chỉ có thể tăng ở bước cuối của Thuật toán 2, nơi nối t_i với $i = 1, \dots, n$ vào đầu phải của w . Do đó việc xóa một đoạn không rỗng chỉ có thể xảy ra nhiều nhất n lần. Tổng thời gian cho thân vòng lặp là $O(n)$, và toàn bộ phần `CANONIZE`, tức phần thời gian thứ nhất, cũng là $O(n)$. $\dashv$

Nhận xét 1. Theo J. Karkkainen, trong dạng cuối cùng, thuật toán này là họ hàng khá gần của phương pháp McCreight [9]. Khác biệt kỹ thuật chính dường như là: mỗi lần thực hiện thân vòng lặp chính của Thuật toán 2 tiêu thụ một ký hiệu văn bản t_i , trong khi mỗi lần thực hiện thân vòng lặp chính của thuật toán McCreight đi qua một liên kết hậu tố và tiêu thụ không hoặc nhiều ký hiệu văn bản.

Nhận xét 2. Không khó tổng quát hóa Thuật toán 2 cho phiên bản động sau của bài toán cây hậu tố, tương tự bài toán khớp từ điển thích nghi trong [2]: duy trì một cây hậu tố tổng quát có kích thước tuyến tính biểu diễn mọi hậu tố của các xâu T_i trong tập $\{T_1, \dots, T_k\}$ dưới các phép chèn hoặc xóa một xâu T_i . Thuật toán thu được thực hiện mỗi cập nhật trong thời gian $O(|T_i|)$.

5 Xây dựng ô-tô-mát hậu tố

Ô-tô-mát hậu tố $SA(T)$ của một xâu $T = t_1 \cdots t_n$ là DFA tối thiểu chấp nhận tất cả hậu tố của T .

Vì $STrie(T)$ là một DFA cho các hậu tố của T , ta có thể thu được $SA(T)$ bằng cách tối thiểu hóa $STrie(T)$ theo cách chuẩn. Tối thiểu hóa hoạt động bằng cách gộp các trạng thái tương đương, tức các trạng thái mà từ đó $STrie(T)$ chấp nhận cùng một tập xâu. Dùng liên kết hậu tố, ta nhận được một đặc trưng tự nhiên cho các trạng thái tương đương như sau.

Một trạng thái s của $STrie(T)$ được gọi là **thiết yếu** nếu có ít nhất hai liên kết hậu tố khác nhau trở tới s , hoặc nếu $s = \overline{t_1 \cdots t_k}$ với một k nào đó.

Định lý 3 Cho s và r là hai trạng thái của $STrie(T)$. Tập xâu được chấp nhận từ s bằng tập xâu được chấp nhận từ r khi và chỉ khi đường liên kết hậu tố bắt đầu từ s chứa r , hoặc đường bắt đầu từ r chứa s , và đường con từ s tới r , hoặc từ r tới s , không chứa trạng thái thiết yếu nào khác ngoài, có thể, chính s hoặc chính r .

Chứng minh. Định lý suy ra từ các quan sát sau.

Tập xâu được chấp nhận từ một trạng thái của $STrie(T)$ là một tập con của các hậu tố của T , nên mỗi xâu được chấp nhận có độ dài khác nhau.

Một xâu độ dài i được chấp nhận từ trạng thái s của $STrie(T)$ khi và chỉ khi đường liên kết hậu tố bắt đầu từ trạng thái $\overline{t_1 \cdots t_{n-i}}$ chứa s .

Các liên kết hậu tố tạo thành một cây có hướng về gốc *root*. $\dashv$

Điều này gợi ý một phương pháp xây dựng $SA(T)$ bằng một Thuật toán 1 đã sửa đổi. Điểm mới là phép xây dựng chỉ nên tạo một trạng thái mới nếu trạng thái đó là thiết yếu. Một trạng thái không thiết yếu s được gộp với trạng thái thiết yếu đầu tiên đứng trước s trên đường liên kết hậu tố đi qua s . Theo Định lý 3, các trạng thái đó là tương đương, nên việc gộp là đúng.

Vì chỉ có $O(|T|)$ trạng thái thiết yếu, thuật toán thu được có thể được cài đặt để chạy trong thời gian tuyến tính. Thuật toán này hóa ra tương tự các thuật toán trong [4, 5, 6], nên bài viết gốc không trình bày chi tiết.

Lời cảm ơn

J. Karkkainen đã chỉ ra một số điểm thiếu chính xác trong phiên bản trước của công trình này [10]. Tác giả cũng cảm ơn E. Sutinen, D. Wood, và đặc biệt là S. Kurtz cùng G. A. Stephen vì nhiều nhận xét hữu ích.

References

- [1] A. Aho and M. Corasick. Efficient string matching: An aid to bibliographic search. *Communications of the ACM*, 18:333–340, 1975.
- [2] A. Amir and M. Farach. Adaptive dictionary matching. In *Proc. 32nd IEEE Annual Symposium on Foundations of Computer Science*, pages 760–766, 1991.
- [3] A. Apostolico. The myriad virtues of subword trees. In A. Apostolico and Z. Galil, editors, *Combinatorial Algorithms on Words*, pages 85–95. Springer-Verlag, 1985.
- [4] A. Blumer et al. The smallest automaton recognizing the subwords of a text. *Theoretical Computer Science*, 40:31–55, 1985.
- [5] M. Crochemore. Transducers and repetitions. *Theoretical Computer Science*, 45:63–86, 1986.
- [6] M. Crochemore. String matching with constraints. In M. P. Chytil, L. Janiga, and V. Koubek, editors, *Mathematical Foundations of Computer Science 1988*, Lecture Notes in Computer Science 324, pages 44–58. Springer-Verlag, 1988.
- [7] Z. Galil and R. Giancarlo. Data structures and algorithms for approximate string matching. *Journal of Complexity*, 4:33–72, 1988.
- [8] M. Kempf, R. Bayer, and U. Guntzer. Time optimal left to right construction of position trees. *Acta Informatica*, 24:461–474, 1987.
- [9] E. McCreight. A space-economical suffix tree construction algorithm. *Journal of the ACM*, 23:262–272, 1976.
- [10] E. Ukkonen. Constructing suffix trees on-line in linear time. In J. van Leeuwen, editor, *Algorithms, Software, Architecture: Information Processing 92*, volume I, pages 484–492. Elsevier, 1992.
- [11] E. Ukkonen. Approximate string-matching over suffix trees. In A. Apostolico, M. Crochemore, Z. Galil, and U. Manber, editors, *Combinatorial Pattern Matching, CPM'93*, Lecture Notes in Computer Science 684, pages 228–242. Springer-Verlag, 1993.

- [12] E. Ukkonen and D. Wood. Approximate string matching with suffix automata. *Algorithmica*, 10:353–364, 1993.
- [13] P. Weiner. Linear pattern matching algorithms. In *IEEE 14th Annual Symposium on Switching and Automata Theory*, pages 1–11, 1973.